Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине «Конструирование программного обеспечения»
Тема: «Настройка взаимодействия с файлами»

Выполнил:
Студент 2 курса
250966
Группы ПО-17
Фещук А.А.
Проверила:
Кулик А.Д.

Брест 2026

**Цель работы:** настроить взаимодействие между проектом, созданным в предыдущей лабораторной работе, и различными файлами на компьютере.

## Ход работы

4. Все параметры хранятся в переменных типа int. Все их возможные значения хранятся в отдельном файле и считываются в контейнеры типа Query (по контейнеру на каждое поле настроек), с которым и происходит работа по изменению текущего значения поля.

```
#include <iostream>
#include <string>
#include <fstream>
#include <queue>

using namespace std;

const int MENU_START_GAME = 1;
const int MENU_LOAD_GAME = 2;
const int MENU_SETTINGS = 3;
const int MENU_ABOUT = 4;
const int MENU_EXIT = 5;

const int SET_CITY_SIZE = 1;
const int SET_BUDGET = 2;
const int SET_POPULATION = 3;
const int SET_BACK = 4;

const string FILE_SETTINGS      = "settings.txt";
const string FILE_CITY_SIZES    = "city_size_values.txt";
const string FILE_BUDGETS       = "budget_values.txt";
const string FILE_POPULATIONS   = "population_values.txt";

struct GameSettings {
    int citySize;
    int budget;
    int population;

    GameSettings() : citySize(10), budget(1000), population(100) {}
};
```

```
class SettingsManager {
public:
    GameSettings settings;

    queue<int> citySizeQueue;
    queue<int> budgetQueue;
    queue<int> populationQueue;

    SettingsManager() {
        ensureValuesFile(FILE_CITY_SIZES,  {10, 20, 30});
        ensureValuesFile(FILE_BUDGETS,     {1000, 5000, 10000});
        ensureValuesFile(FILE_POPULATIONS, {100, 500, 1000});

        loadQueueFromFile(FILE_CITY_SIZES,   citySizeQueue);
        loadQueueFromFile(FILE_BUDGETS,      budgetQueue);
        loadQueueFromFile(FILE_POPULATIONS,  populationQueue);

        if (!loadSettings()) {
            saveSettings();
        }
    }

    void switchCitySize() {
        settings.citySize = nextFromQueue(citySizeQueue,
settings.citySize);
        saveSettings();
    }

    void switchBudget() {
        settings.budget = nextFromQueue(budgetQueue, settings.budget);
        saveSettings();
    }

    void switchPopulation() {
        settings.population = nextFromQueue(populationQueue,
settings.population);
        saveSettings();
    }

private:
```

```
    void ensureValuesFile(const string& filename, const
initializer_list<int>& values) {
        ifstream check(filename);
        if (check.good()) return;
        check.close();

        ofstream fout(filename);
        for (int v : values) fout << v << "\n";
    }

    void loadQueueFromFile(const string& filename, queue<int>& q) {
        ifstream fin(filename);
        int value;
        while (fin >> value) q.push(value);
    }

    int nextFromQueue(queue<int>& q, int current) {
        if (q.empty()) return current;

        int first = q.front();
        int next = first;
        bool found = false;
        int size = (int)q.size();

        for (int i = 0; i < size; ++i) {
            int v = q.front();
            q.pop();
            q.push(v);
            if (!found && v == current) {
                found = true;
                if (i + 1 < size) {
                    next = q.front();
                } else {
                    next = first;
                }
            }
        }
        if (!found) next = first;
        return next;
    }
```

```
    bool loadSettings() {
        ifstream fin(FILE_SETTINGS);
        if (!fin.is_open()) return false;

        int cs, b, p;
        if (fin >> cs >> b >> p) {
            settings.citySize   = cs;
            settings.budget     = b;
            settings.population = p;
            return true;
        }
        return false;
    }

    void saveSettings() {
        ofstream fout(FILE_SETTINGS);
        fout << settings.citySize   << "\n"
             << settings.budget     << "\n"
             << settings.population << "\n";
    }
};

void clearScreen() {
#ifdef _WIN32
    system("cls");
#else
    system("clear");
#endif
}

void waitForEnter() {
    cout << "\nНажмите Enter, чтобы вернуться в меню...";
    cin.ignore(10000, '\n');
    cin.get();
}

void showMainMenu() {
    cout << "====================================\n";
    cout << "          ГОРОД МЕЧТЫ\n";
```

```
    cout << "====================================\n";
    cout << MENU_START_GAME << ". Начать игру\n";
    cout << MENU_LOAD_GAME << ". Загрузить игру\n";
    cout << MENU_SETTINGS << ". Настройки\n";
    cout << MENU_ABOUT << ". О программе\n";
    cout << MENU_EXIT << ". Выход\n";
    cout << "====================================\n";
    cout << "Выберите пункт меню: ";
}

void startGame() {
    clearScreen();
    cout << "=== НАЧАЛО ИГРЫ ===\n";
    cout << "Вы начали новую игру в «Город мечты»!\n";
    waitForEnter();
}

void loadGame() {
    clearScreen();
    cout << "=== ЗАГРУЗКА ИГРЫ ===\n";
    cout << "Данная функция на данный момент недоступна\n";
    cout << "и появится в ближайших обновлениях.\n";
    waitForEnter();
}

void showSettingsMenu(const SettingsManager& m) {
    cout << "====================================\n";
    cout << "             НАСТРОЙКИ\n";
    cout << "====================================\n";
    cout << SET_CITY_SIZE  << ". Размер города:      " <<
m.settings.citySize   << "\n";
    cout << SET_BUDGET     << ". Начальный бюджет:   " <<
m.settings.budget     << "\n";
    cout << SET_POPULATION << ". Количество жителей: " <<
m.settings.population << "\n";
    cout << SET_BACK       << ". Назад\n";
    cout << "====================================\n";
    cout << "Выберите пункт: ";
}

void settingsMenu(SettingsManager& m) {
```

```
    int choice = 0;
    while (choice != SET_BACK) {
        clearScreen();
        showSettingsMenu(m);
        cin >> choice;

        if (cin.fail()) {
            cin.clear();
            cin.ignore(10000, '\n');
            continue;
        }

        switch (choice) {
        case SET_CITY_SIZE:
            m.switchCitySize();
            break;
        case SET_BUDGET:
            m.switchBudget();
            break;
        case SET_POPULATION:
            m.switchPopulation();
            break;
        case SET_BACK:
            break;
        default:
            cout << "Неверный пункт меню!\n";
            waitForEnter();
            break;
        }
    }
}

void showAbout() {
    clearScreen();
    cout << "=== О ПРОГРАММЕ ===\n";
    cout << "Игра: «Город мечты»\n";
    cout << "Версия программы: 1.1\n";
    cout << "Студент: Фещук Артём\n";
    cout << "\n--- Рекомендация музыкального плейлиста ---\n";
    cout << "Для атмосферы строительства города советуем:\n";
```

```
    cout << "  • «City Builder OST» — спокойный эмбиент\n";
    cout << "  • «SimCity Soundtrack» — классика градостроения\n";
    cout << "  • «Lo-Fi City Beats» — ненавязчивый фон для игры\n";
    cout << "  • «Minecraft playlist»\n";
    cout << "Приятной игры под хорошую музыку!\n";
    waitForEnter();
}

int main() {
    setlocale(LC_ALL, "Russian");

    SettingsManager manager;   // локальная переменная, без
глобальной
    int choice = 0;

    while (choice != MENU_EXIT) {
        clearScreen();
        showMainMenu();
        cin >> choice;

        if (cin.fail()) {
            cin.clear();
            cin.ignore(10000, '\n');
            continue;
        }

        switch (choice) {
        case MENU_START_GAME:
            startGame();
            break;
        case MENU_LOAD_GAME:
            loadGame();
            break;
        case MENU_SETTINGS:
            settingsMenu(manager);
            break;
        case MENU_ABOUT:
            showAbout();
            break;
        case MENU_EXIT:
```

```
            cout << "Выход из программы...\n";
            break;
        default:
            cout << "Неверный пункт меню!\n";
            waitForEnter();
            break;
        }
    }

    return 0;
}
```

**Вывод:** настроил взаимодействие между проектом, созданным в предыдущей лабораторной работе, и различными файлами на компьютере.